Ex7: Model Planning and Building - Clustering

AIM: To implement the K-Means clustering algorithm on the Mall Customer Segmentation dataset for customer segmentation by exploring data, standardizing key features, identifying the optimal number of clusters using the Elbow Method, visualizing cluster distributions and centroids, and analyzing cluster statistics.

ALGORITHM:

1. Import necessary Python libraries (pandas, numpy, matplotlib.pyplot, seaborn, StandardScaler from sklearn.preprocessing, and KMeans from sklearn.cluster).
2. Load the Mall Customer Segmentation Dataset from Kaggle into a Pandas DataFrame.
3. Explore the dataset by displaying the first and last five records, checking shape and data types, identifying missing values and duplicate records, and displaying summary statistics.
4. Select the target features for clustering: Annual Income (k$) and Spending Score (1-100).
5. Standardize the selected feature columns using StandardScaler.
6. Visualize the unscaled/scaled features using Scatter Plots and Pair Plots.
7. Determine the optimal number of clusters (K) using the Elbow Method by plotting the Elbow Curve (within-cluster sum of squares vs. K).
8. Train the K-Means Clustering model using the optimal value of K.
9. Assign cluster labels to each customer, append the labels as a new column in the DataFrame, and save the resulting dataset as a file.
10. Visualize customer clusters on a scatter plot (X-axis: Annual Income, Y-axis: Spending Score) using unique colors for each cluster, highlighting cluster centroids, and displaying centroid coordinates.
11. Perform cluster analysis by calculating customer counts per cluster and computing the average Annual Income and Spending Score for each segment.

EX7 – Model Planning and Building: Clustering

1. Load the Mall Customer Segmentation Dataset into a DataFrame

```
#Exp_No_7
#Sanjay_S_241801246
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("Mall_Customers.csv")
print(df.head())
```

```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
```

2. Explore the Dataset

2.1 Display the First and Last Five Records

```
#Exp_No_7
#Sanjay_S_241801246
print("First Five Records:")
print(df.head())
print("\nLast Five Records:")
print(df.tail())
```

```
First Five Records:
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Last Five Records:
     CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
195         196  Female   35                 120                      79
196         197  Female   45                 126                      28
197         198    Male   32                 126                      74
198         199    Male   32                 137                      18
199         200    Male   30                 137                      83
```

2.2 Check Dataset Shape and Data Types

```
#Exp_No_7
#Sanjay_S_241801246
print("Dataset Shape:", df.shape)
```

```
Dataset Shape: (200, 5)
```

```
print("\nData Types:")
print(df.dtypes)
```

```
Data Types:
CustomerID                int64
Gender                   object
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object
```

```
print("\nDataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

2.3 Identify Missing and Duplicate Records

```
print("Missing Values:")
print(df.isnull().sum())
print("\nDuplicate Records:", df.duplicated().sum())
```

```
Missing Values:
CustomerID              0
Gender                  0
Age                     0
Annual Income (k$)      0
Spending Score (1-100)  0
dtype: int64

Duplicate Records: 0
```

2.4 Display Summary Statistics

```
print(df.describe(include="all"))
```

```
        CustomerID  Gender         Age  Annual Income (k$)  \
count   200.000000     200  200.000000          200.000000
unique         NaN       2         NaN                 NaN
top            NaN  Female         NaN                 NaN
freq           NaN     112         NaN                 NaN
mean    100.500000     NaN   38.850000           60.560000
std      57.879185     NaN   13.969007           26.264721
min       1.000000     NaN   18.000000           15.000000
25%      50.750000     NaN   28.750000           41.500000
50%     100.500000     NaN   36.000000           61.500000
75%     150.250000     NaN   49.000000           78.000000
max     200.000000     NaN   70.000000          137.000000

        Spending Score (1-100)
count               200.000000
unique                     NaN
top                        NaN
freq                       NaN
mean                 50.200000
std                  25.823522
min                   1.000000
25%                  34.750000
50%                  50.000000
75%                  73.000000
max                  99.000000
```

3. Select Important Features for Clustering

```
#Exp_No_7
#Sanjay_S_241801246
X = df[["Annual Income (k$)", "Spending Score (1-100)"]]
print(X.head())
```

```
   Annual Income (k$)  Spending Score (1-100)
0                  15                      39
1                  15                      81
2                  16                       6
3                  16                      77
4                  17                      40
```

4. Standardize the Selected Features Using StandardScaler

```
#Exp_No_7
#Sanjay_S_241801246
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Standardized Features:")
print(X_scaled[:5])
```

```
Standardized Features:
[[-1.73899919 -0.43480148]
 [-1.73899919  1.19570407]
 [-1.70082976 -1.71591298]
 [-1.70082976  1.04041783]
 [-1.66266033 -0.39597992]]
```

5. Visualize the Selected Features

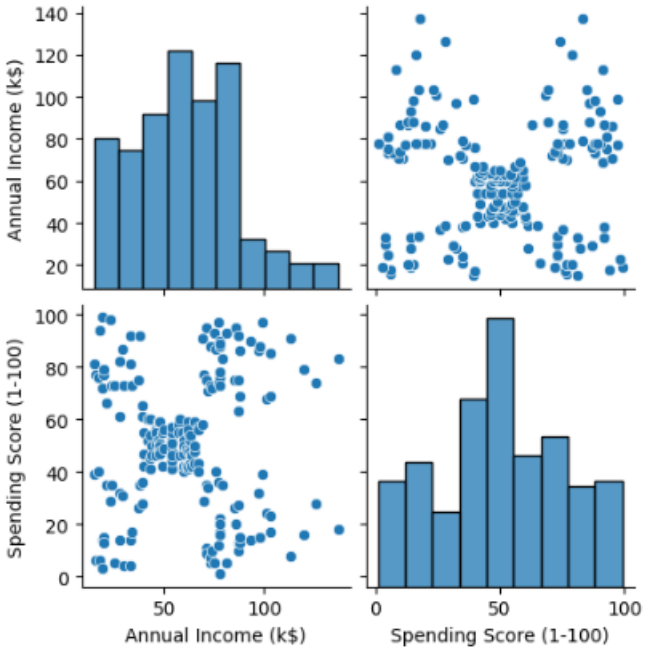
5.1 Scatter Plot

```
#Exp_No_7
#Sanjay_S_241801246
plt.figure(figsize=(8, 5))
plt.scatter(X["Annual Income (k$)"], X["Spending Score (1-100)"])
plt.title("Annual Income vs Spending Score")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.show()
```



5.2 Pair Plot

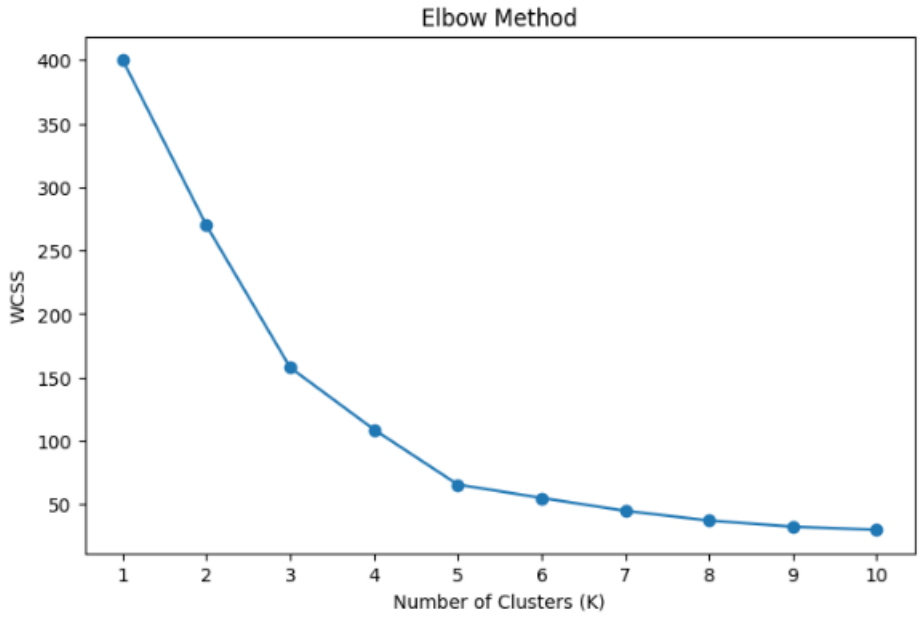
```
#Exp_No_7
#Sanjay_S_241801246
sns.pairplot(
    df[["Annual Income (k$)", "Spending Score (1-100)"]]
)
plt.show()
```



Model Planning and Building

6. Determine the Optimal Number of Clusters Using the Elbow Method

```
#Exp_No_7
#Sanjay_S_241801246
from sklearn.cluster import KMeans
wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker="o")
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.xticks(range(1, 11))
plt.show()
```



7. Train K-Means, Assign Cluster Labels, and Save the Dataset

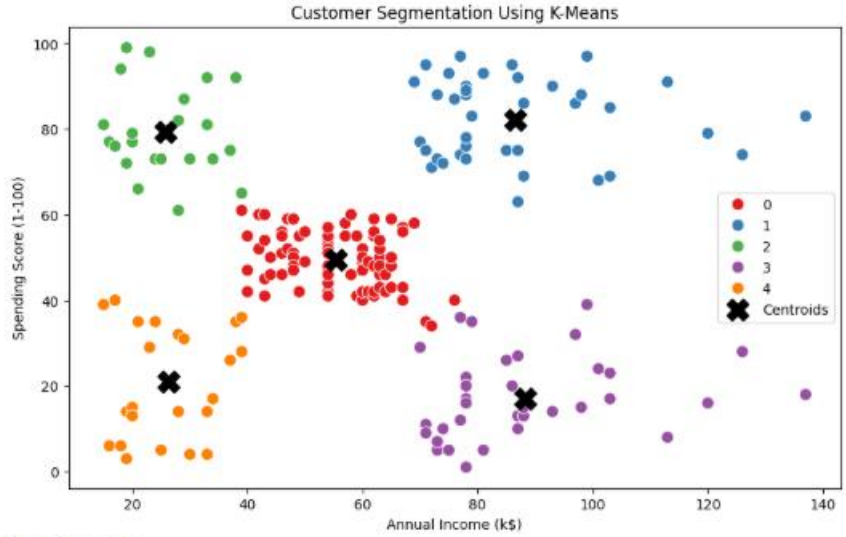
```
#Exp_No_7
#Sanjay_S_241801246
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X_scaled)
print(df.head())
df.to_csv("Mall_Customers_Clustered.csv", index=False)
print("Clustered dataset saved successfully.")
```

```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)  \
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

   Cluster
0        4
1        2
2        4
3        2
4        4
Clustered dataset saved successfully.
```

8. Visualize Customer Clusters and Display Centroids

```
#Exp_No_7
#Sanjay_S_241801246
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x="Annual Income (k$)",
    y="Spending Score (1-100)",
    hue="Cluster",
    palette="Set1",
    s=80
)
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker="X",
    s=250,
    color="black",
    label="Centroids"
)
plt.title("Customer Segmentation Using K-Means")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.legend()
plt.show()
print("Cluster Centroids:")
print(pd.DataFrame(
    centroids,
    columns=["Annual Income (k$)", "Spending Score (1-100)"]
))
```

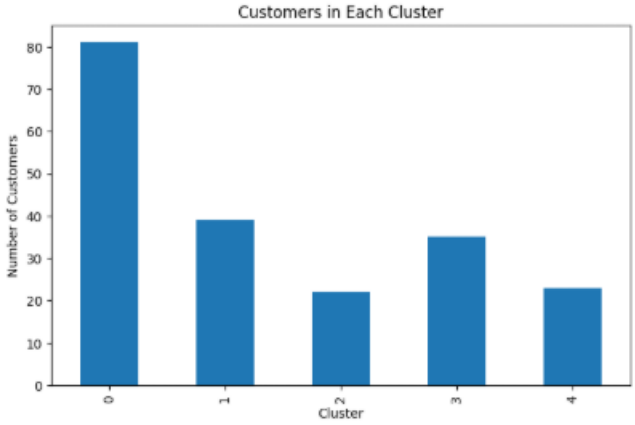



```
Cluster Centroids:
   Annual Income (k$)  Spending Score (1-100)
0           55.296296               49.518519
1           86.538462               82.128205
2           25.727273               79.363636
3           88.200000               17.114286
4           26.304348               20.913043
```

9. Calculate the Number of Customers in Each Cluster

```
#Exp_No_7
#Sanjay_S_241801246
cluster_counts = df["Cluster"].value_counts().sort_index()
print("Number of Customers in Each Cluster:")
print(cluster_counts)
cluster_counts.plot(kind="bar", figsize=(8, 5))
plt.title("Customers in Each Cluster")
plt.xlabel("Cluster")
plt.ylabel("Number of Customers")
plt.show()
```

```
Number of Customers in Each Cluster:
Cluster
0    81
1    39
2    22
3    35
4    23
Name: count, dtype: int64
```



10. Compare Average Annual Income and Spending Score for Each Cluster

```
#Exp_No_7
#Sanjay_S_241801246
cluster_analysis = df.groupby("Cluster")[
    ["Annual Income (k$)", "Spending Score (1-100)"]
].mean()
print("Average Income and Spending Score by Cluster:")
print(cluster_analysis)
```

```
Average Income and Spending Score by Cluster:
         Annual Income (k$)  Spending Score (1-100)
Cluster
0                 55.296296               49.518519
1                 86.538462               82.128205
2                 25.727273               79.363636
3                 88.200000               17.114286
4                 26.304348               20.913043
```

RESULT: The K-Means clustering algorithm was successfully implemented for customer segmentation on the Mall Customer Segmentation dataset. Feature standardization, optimal cluster selection via the Elbow Method, model fitting, centroid visualization, and cluster summary statistics were completed.